

Final Project

Choose One Real-World Concurrent Application

Concurrency, Parallelism & Distributed Systems

Lebanese University — Faculty of Engineering — Semester VIII — Academic Year 2025–2026

Instructor: Dr. Mohamad Aoude | Group project: 3–4 students | Final weight: 30 %

Overview

Each group must choose **one** project topic from the list below and build one real-life application in that topic. The topics are the same project directions used in the course file: web/Internet systems, machine-learning pipelines, transaction/order processing, and IoT/sensor ingestion.

Choose one topic only.

All groups follow the same concurrency and distributed-systems grading requirements, but each group implements only one selected application topic. The project must be real enough to need concurrency; a simple CRUD application does not satisfy the assignment.

1. Project Topics

1.1. Topic A — Real-Time Web or Internet Backend

Example applications: live chat server, collaborative whiteboard, real-time notification service, online multiplayer lobby, or WebSocket-based customer-support system.

Why it needs this course: many clients connect at the same time; messages must be delivered without races; slow clients must not block fast clients; the server must survive bursts, disconnects, and partial failures.

Minimum scope:

- HTTP, WebSocket, TCP, or gRPC server handling at least 50 concurrent clients.
- Per-room, per-channel, or per-user message ordering without one global lock.
- Bounded queues and backpressure for slow clients.
- Graceful handling of disconnects and timeouts.
- Metrics for throughput, latency, active clients, dropped messages, and errors.

1.2. Topic B — Machine-Learning or Data-Analysis Pipeline

Example applications: sentiment-analysis API, image classifier, document classifier, recommendation scorer, anomaly detector, or OCR-style document intake service.

Why it needs this course: requests arrive concurrently; pre-processing, inference, and post-processing have different costs; CPU-bound work may benefit from parallelism; Python services may be limited by the GIL; slow model calls need timeouts and fallbacks.

Minimum scope:

- API accepting concurrent analysis requests.
- Pipeline with at least three stages: validate/pre-process, infer/analyze, post-process/store.
- Bounded queue between stages or explicit async composition.
- Sequential vs parallel benchmark for one CPU-bound operation.
- If Python is used, explain threads vs processes vs separate service using the Week 9 GIL material.

1.3. Topic C — Distributed Order or Transaction Processing

Example applications: e-commerce order system, food-delivery dispatcher, ticket booking system, appointment booking, inventory reservation, or payment confirmation workflow.

Why it needs this course: orders arrive concurrently; inventory or booking slots are shared state; retries must not duplicate orders; payment and warehouse services may be slow or unavailable.

Minimum scope:

- Concurrent order or booking submission.
- Correct shared-state update: no overselling, double booking, or duplicate confirmation.
- Idempotency key for retried requests.
- Event log or queue for state transitions such as placed, confirmed, failed, shipped.
- Simulated downstream failure with timeout, retry, and recovery behavior.

1.4. Topic D — IoT or Sensor Data Ingestion Pipeline

Example applications: smart-building monitor, vehicle telemetry aggregator, environmental sensor dashboard, energy-usage monitor, or health device event collector.

Why it needs this course: many producers publish at the same time; ingestion must be non-blocking; aggregation must avoid races; overload must be handled explicitly; a failed sensor must not stop the pipeline.

Minimum scope:

- At least 20 simulated sensor producers running concurrently.
- Bounded ingestion queue with measured lag under overload.
- Thread-safe aggregation using atomics, locks, immutable snapshots, or message ownership.
- Alert subsystem that reacts to threshold crossings.
- Graceful degradation policy when the system is full.

2. How the Project Matches the Syllabus

Whichever topic is chosen, the project must apply the same course mechanisms:

- **Weeks 1–2:** concurrent requests, threads, lifecycle, coordination, and when not to add concurrency.
- **Week 3:** race conditions, synchronization, monitors, locks, and deadlock avoidance.
- **Week 4:** visibility, happens-before, volatile, immutability, and safe publication.
- **Week 5:** `ExecutorService`, bounded queues, backpressure, atomics, and read-write locks.
- **Week 6:** Fork/Join, task granularity, break-even measurement, and parallel-stream pitfalls.
- **Week 7:** `CompletableFuture`, async pipelines, timeouts, fallbacks, and stress testing.
- **Week 9:** Python GIL and runtime limits when a Python component is used.
- **Weeks 10–14:** network failure, serialization boundaries, Kafka or equivalent event logs, idempotency, tracing, and failure recovery.

3. Required Technical Features

Every selected topic must include:

- Java 21 as the main project language.

- A real concurrent entry point: HTTP, WebSocket, TCP, gRPC, or simulated producers.
- Explicit thread pool or executor with documented sizing rationale.
- At least one bounded queue or bounded executor.
- Correct protection of shared mutable state.
- At least one asynchronous pipeline or fan-out/fan-in operation.
- At least one timeout and fallback path.
- Metrics for throughput, latency, queue depth, completed work, failed work, and rejected/delayed work.
- Graceful shutdown.

Unbounded resources are not accepted.

The project may not use unbounded thread creation or unbounded queues as the main design. Overload behavior must be intentional and measurable.

4. Distributed Final Version

The final version must include at least one real network boundary. Examples:

- API service calling a worker service.
- Web server publishing events to a broker and worker consuming them.
- Sensor gateway sending data to an aggregation service.
- Order API calling payment/inventory services.
- Java service calling a Python ML service.

The network boundary must include serialization, timeout behavior, logging, and failure handling. A method call inside one process is not enough for the final distributed requirement.

5. Required Evidence

The final submission must include:

- source code that builds and runs from a clean checkout;
- README.md with run commands, tests, and failure-injection instructions;
- load-test table with throughput and latency;
- stress-test output for concurrency correctness;
- logs for one successful request/event and one failed/recovered request/event;
- screenshots or terminal output from failure injection;
- architecture diagram, async sequence diagram, and failure propagation diagram.

Concurrency Scorecard

Question	Mechanism used	Evidence
Thread-safe?		
Visibility guaranteed?		
Deadlock-free?		
Liveness guaranteed?		
Bounded resources?		
Failure recovery path?		

6. Architecture Decision Memo

Each group must submit a 1–2 page memo about one important design decision: queue capacity, worker pool size, locking strategy, REST vs gRPC vs sockets, Kafka vs in-memory queue, Java vs Python worker, or retry/idempotency policy.

The memo must answer:

- Q1.** What guarantee does your chosen design provide?
- Q2.** What failure modes does it prevent?
- Q3.** What failure modes does it introduce?
- Q4.** How does it behave under overload? Give measured numbers.
- Q5.** How would a new engineer debug it?

7. Final Presentation

Each group has **10 minutes**: 8 minutes presentation and 2 minutes questions. Every group member must speak.

1. Show the selected topic and real application demo.
2. Identify threads, queues, shared state, and network boundaries.
3. Show one failure you injected and explain how the system survived.
4. Show logs, metrics, or traces as evidence.
5. Explain one tradeoff your group made.

8. Grading Summary

Component	Weight within final project
Working selected application topic	25 %
Concurrency correctness and stress evidence	25 %
Async design, bounded resources, and measurement	20 %
Distributed failure handling, idempotency, and tracing	20 %
Presentation and individual explanation	10 %
Total	100 %

Final course weight: this project is the Week 14 final checkpoint and counts for **30 %** of the course grade, consistent with the syllabus.

9. Technical Constraints

- **Group size:** 3–4 students.
- **Primary language:** Java 21.
- **Build system:** Gradle.
- **Communication:** HTTP, WebSocket, gRPC, raw TCP sockets, Kafka, or equivalent.
- **Failure injection:** Toxiproxy, Docker, controlled process kill, artificial latency, or instructor-approved script.
- **No required cloud services:** the full project must run locally or with Docker.

- **External APIs:** allowed only as optional enrichment; the core demo must work without Internet access.

On AI-assisted code generation:

AI tools may be used for boilerplate, but every group member must understand the final code. During the presentation, any member may be asked to explain the concurrency design, a failure path, or a specific implementation detail.

10. Submission Checklist

- ☐ Chosen topic clearly stated: A, B, C, or D
- ☐ Source code builds from a clean checkout
- ☐ `README.md` explains setup, run commands, tests, and failure injection
- ☐ Concurrent entry point implemented
- ☐ Bounded queue or bounded executor demonstrated
- ☐ Shared state list with protection mechanisms
- ☐ Stress tests for concurrency correctness
- ☐ Sequential vs parallel benchmark for one CPU-bound step
- ☐ p50/p95/p99 latency and throughput table
- ☐ Timeout and fallback evidence
- ☐ Idempotency or duplicate-event handling where retries are possible
- ☐ Request/event tracing across components
- ☐ Two injected failure scenarios
- ☐ Architecture diagram
- ☐ Async sequence diagram
- ☐ Failure propagation diagram
- ☐ Completed concurrency scorecard
- ☐ Architecture decision memo
- ☐ Presentation slides or demo script

The final project is not about adding threads to an application.

It is about proving that a real concurrent system remains correct, bounded, observable, and recoverable under load and failure.

— Dr. Mohamad Aoude